



CCNP Lab: Host AI Locally



By: Munir, Waseem

Purpose:

The purpose of this lab was to understand how LLM's work, as well as how to locally run them. This lab also included creating a local WebUI for a downloaded AI model, and how to run one off of Docker.

Background:

Large Language Models have shifted from being hosted on large and bulky cloud servers to something that can be run on a laptop. Modern computers, specifically ones with a large amount of vRAM (Video Random Access Memory) are now capable of doing this. As a result, people have begun running complex natural language processing tasks without relying on the internet or remote servers. The major benefit of this is that people now have much stronger control over the environment and the data being processed and how it's being processed.

The core of how strong or natural an LLM is it's parameters. Parameters are often measured into the billions and is simply how nuanced the responses can be. The more parameters a computer has, the better the responses, but it also comes with slower computation and higher memory usage. Most models that are run on desktop setups are in the 3 billion to 8 billion parameter range since they especially with the right one, it can show great performance with the added benefit of not demanding too much processing power.

Once an LLM is trained, it uses something called inference to generate or analyze text. The process of using inference locally requires enough CPU/GPU resources and RAM/VRAM to handle the computations. This is important because OpenAI, Llama and other AI companies have data centers to run their AI, whilst most people only have their local machine to run theirs.

There are several software options for hosting these models locally. One option is LM Studio, which runs the model as a server on your machine. You can then interact with the AI and control how it works through its developer interface. This gives lots of flexibility because other coding languages such as python can be used to send queries and process responses.

Other platforms such as Ollama can be used in Linux environments, but also by setting it up on Windows via Docker. Docker is a tool which creates containers that are similar to lightweight virtual machines. They allow Linux software to run on Windows without changing your main system and they make it easy to setup and reproduce environments. Knowing how to use Docker is very useful for modern development, deployment tasks and hosting models. Linux is an open-source operating system which is widely used for running AI models. It's able to be configured much more than any other operating system, and as a result offered flexibility, stability and a lot of command-line tools which allows it to deploy and manage AI workloads especially when paired with Docker and Ubuntu.

Ubuntu is a very popular Linux distribution, and is often used for development, cloud computing and AI projects. Using ubuntu to run an AI gives access to lots of tools and ensures compatibility with many frameworks. When combined together, Docker, Ollama, and Ubuntu allow a user to deploy an LLM in a controlled environment that can be accessed within the local network. This is like deployments where end-users interact with the model remotely, while you retain control of the server.

The main way to download a LLM is by downloading a .gguf file particularly from websites like HuggingFace. Most open-source models are distributed in this format due to their newer and more efficient format for storing LLM weights. Once a model is loaded, you can manually configure the AI through different parameters like temperature. Temperature controls the randomness in generation. For example, a low temperature would produce straight-forward reasonable answers, while a higher temperature would see more creative/diverse answers that are not usually expected from an AI.

A huge part of this lab was using a virtual environment in our python project. This creates an isolated space so that different projects can have their own versions of libraries downloaded without conflicting with each other. Python, which was the main coding language used in this lab, is the most popular coding language and the most dominant language in the AI space due to its simplicity and speed. The vast number of libraries, including machine learning libraries available with python, make it a high-level coding language. One tool used in this lab was Gradio, a Python library which lets you quickly create web-based interfaces for AI models. This makes it easy to build simple demonstrations or WebUIs without the need to write a full web application.

Throughout this lab, we accessed AIs through an IP Address and a port number. A port number is a virtual communication endpoint on a computer which allows many services to run on the same machine. Let's say you host an AI locally, it may run on port 7860 or 5000, which tells your machine what service to connect to. In the early days of the internet, people had to manually type a websites ip address followed by its port number into a browser. An IP Address is the numbers which identify the machine on the network, similar to an ID for a person, it would be unique to each machine on the network. Because most people cannot remember strings of numbers for each website they want to go to, DNS was created. DNS is like a phonebook for the internet, which translates human-readable domain names into IP Addresses behind the scenes. It's very rare to type an IP Address into a browser because DNS handles the complexity for you and makes web access easier for the average user.

Throughout this lab, we used two different AI's, the first one is LLaMa (Large Language Model Meta AI) and the second one is GPT (generative Pre-trained Transformer). LLaMa is a family of LLM's made by Meta which are built to be smaller but more efficient compared to LLM's while still delivering high performance. Oftentimes they're available in different sizes for users to choose between computational requirements and capabilities. They're deployed in environments where privacy and control over the model are important. GPT is a class of models developed by OpenAI. They're trained on huge datasets to understand and generate human-like text. GPT uses transformer

architecture which lets it capture context and meaning across large spans of text, making its output coherent and relevant.

When local models run on local servers, it can be accessed from the same machine or from other devices on the same network. This generally means that you can host a model on your machine/desktop, then whilst being on the same LAN, access the same AI from a different device. Usually, the model runs on a specific port, and by disabling the computers firewalls, other devices can start to access the AI and use it whilst it's locally running on your machine.

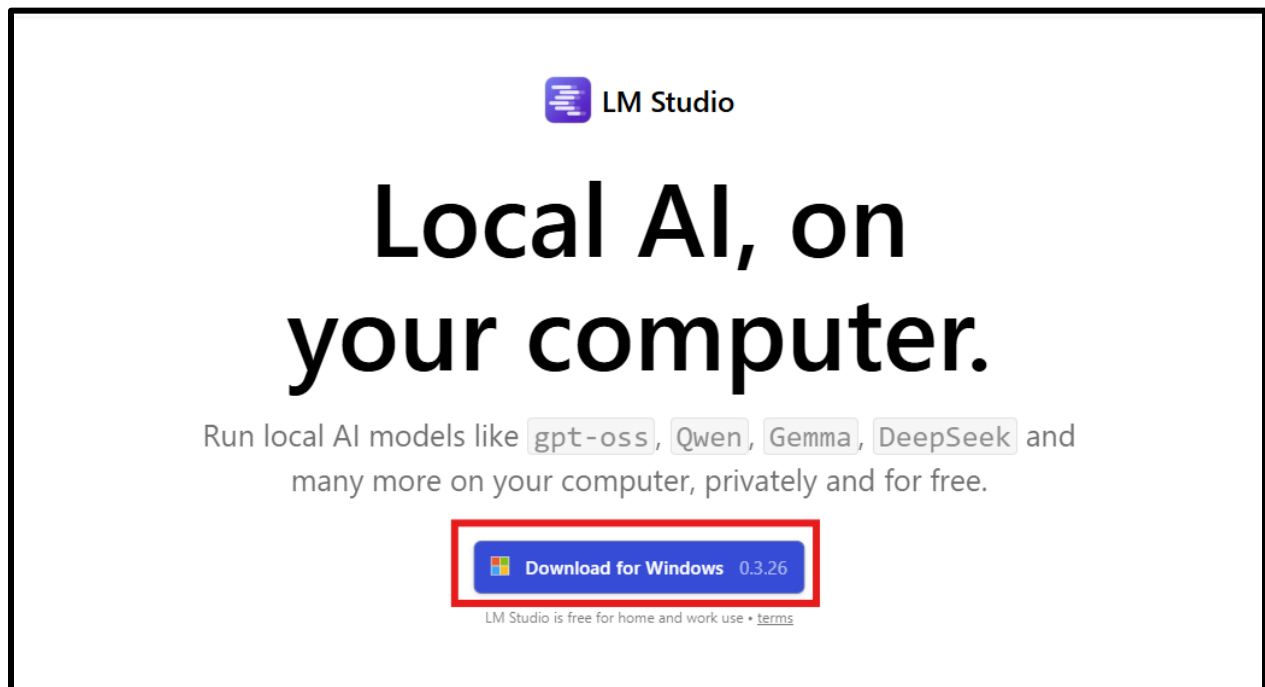
Lab Summary:

In this lab I locally ran an LLM using three different ways. First, I downloaded LM Studio and a gpt GGUF file, and I ran it directly on LM Studio. Second, I manipulated LM Studio's developer interface to allow me to host the AI on my machine as a server, such that I could create a python web interface so that my phone can easily access it. The last way was done by downloading Docker Desktop, and by using Ollama's Linux UI, I was able to run Linux on top of Windows and setup Ollama locally that way.

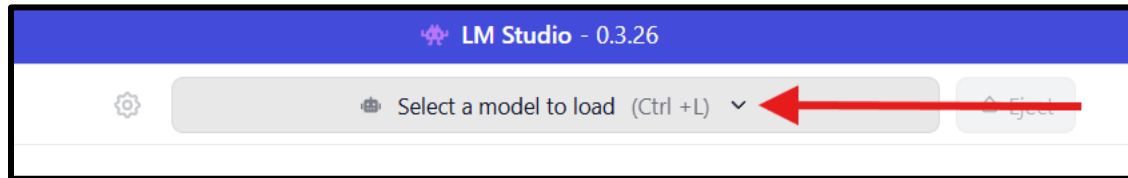
Configurations:

Lab Part 1: Running GPT on LM Studio

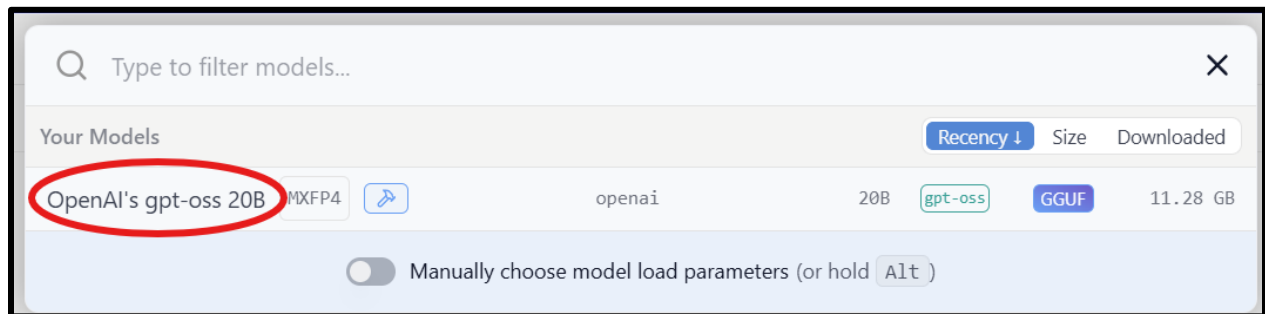
1. Download LM Studio: <https://lmstudio.ai/>



2. Press on “Select a model to load”



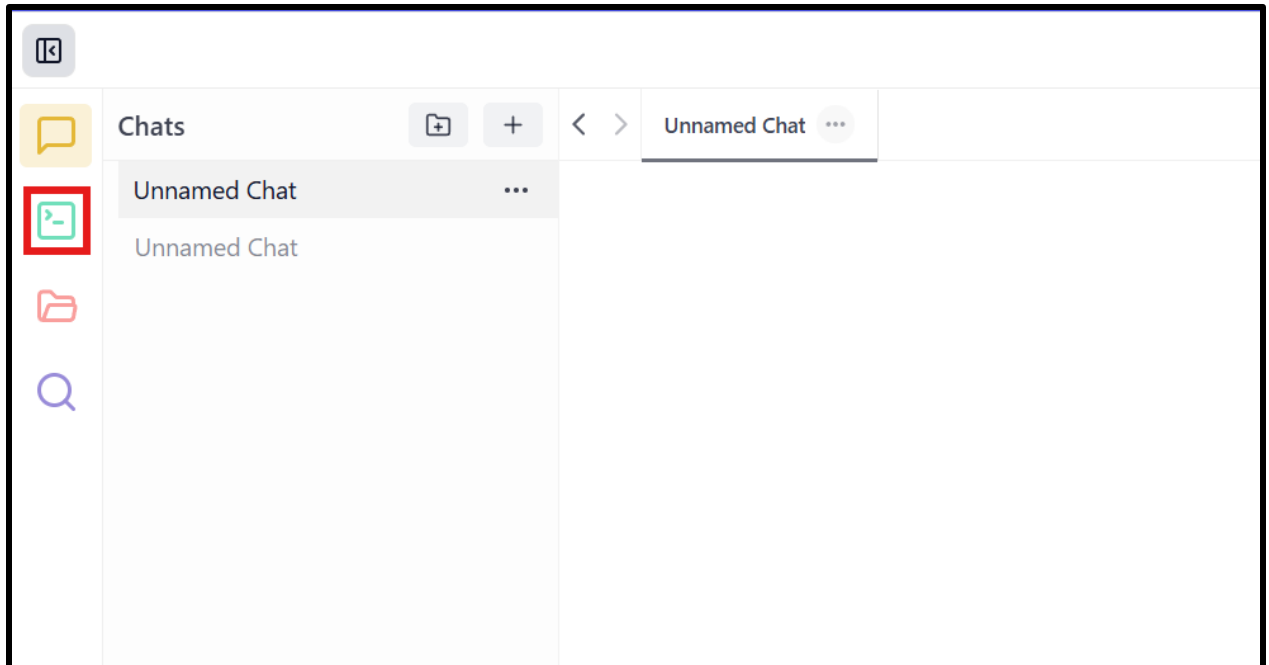
3. Download a model of your choice, likely GTP 20b



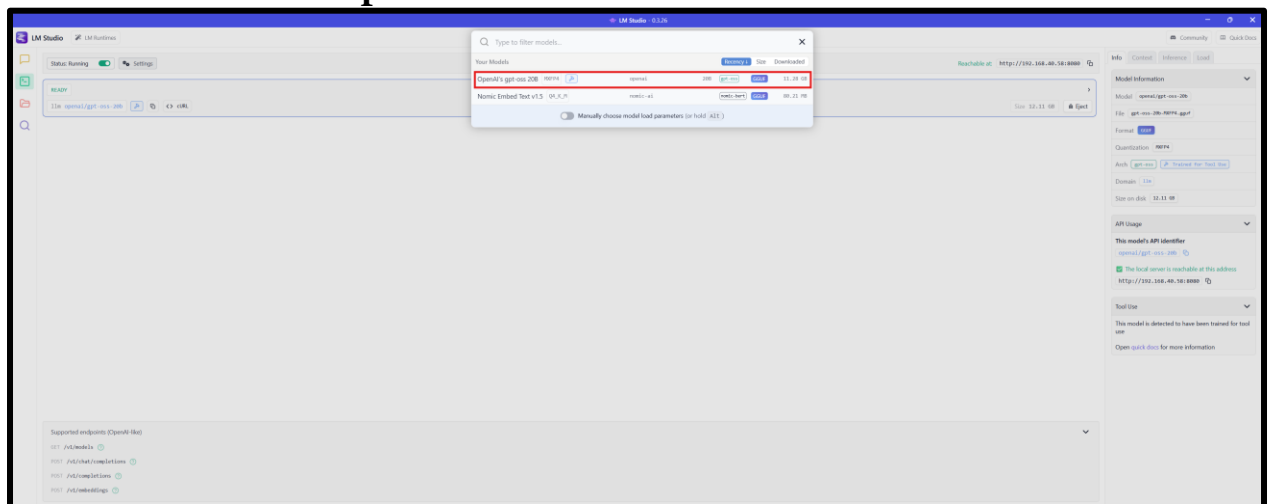
4. After it's finished downloading, you should be able to chat with the AI.

Lab Part 2: Create WebUI using Developer Interface (LMStudio)

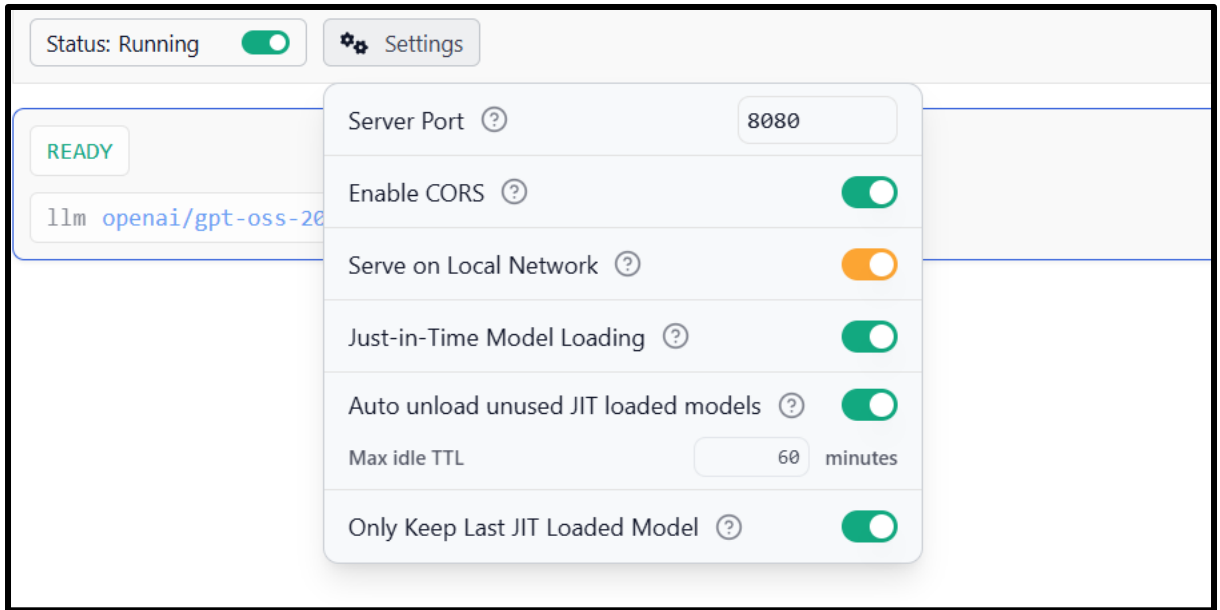
1. Click Developer Interface



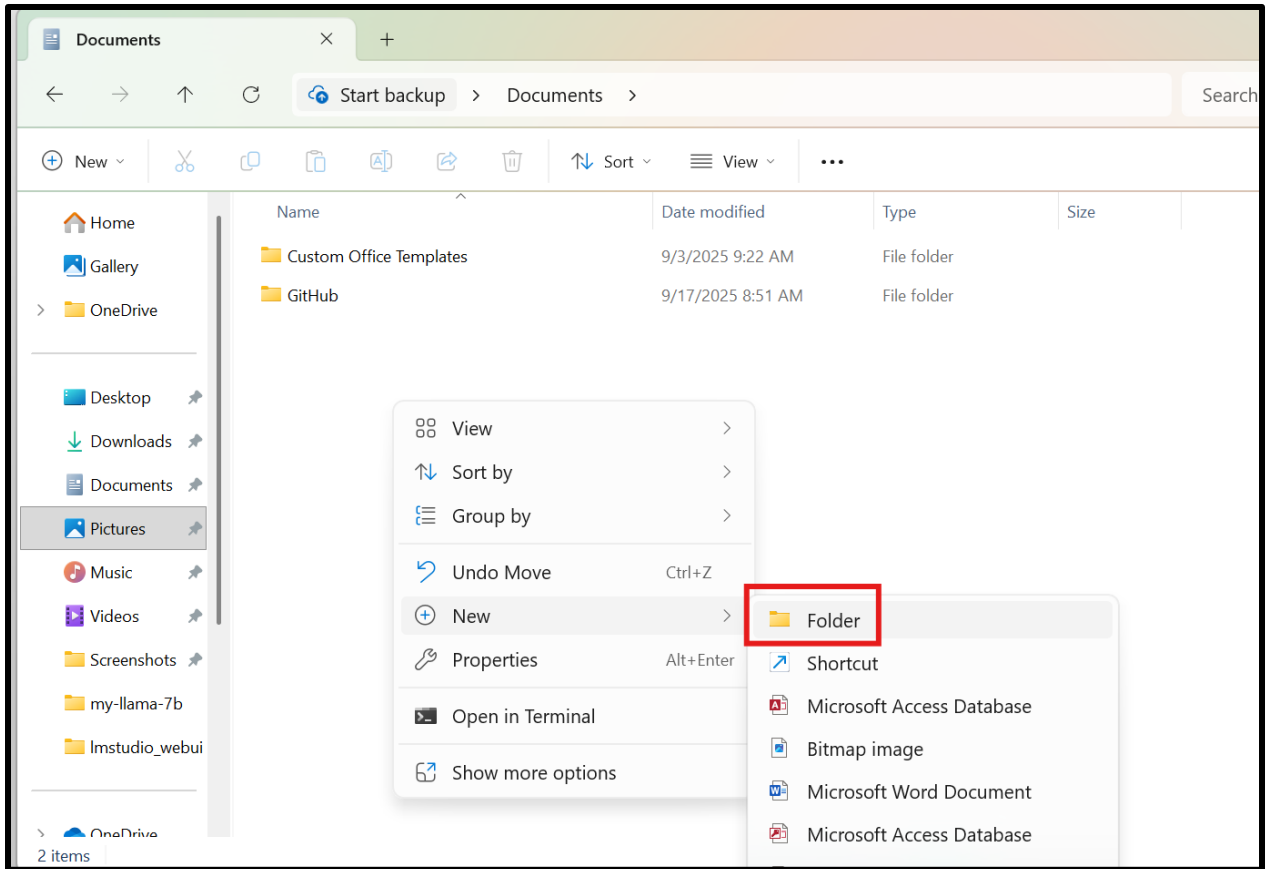
2. Load Model in Developer Interface



3. Change Settings as shown in photo below



4. Create a folder called “lmstudio_webui”.



5. Create and activate a python virtual environment

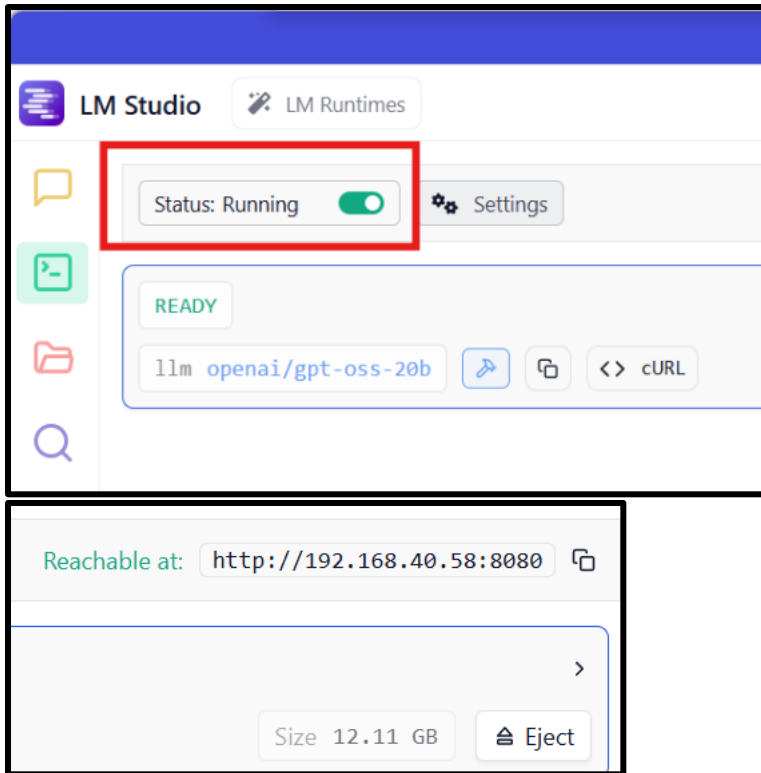
```
python -m venv venv  
venv\Scripts\activate.bat  
pip install gradio requests
```

Note: The first line tells Python run a venv module, which creates a virtual environment. The second venv written is simply the name of the folder for the virtual environment, it could be named anything.

The second line activates the virtual environment, which tells the Python interpreter to only use the libraries inside the venv folder, not the PC.

The third line downloads gradio (Explained in Background), and requests, which helps for making HTTP requests.

6. Start the LM Studio Server



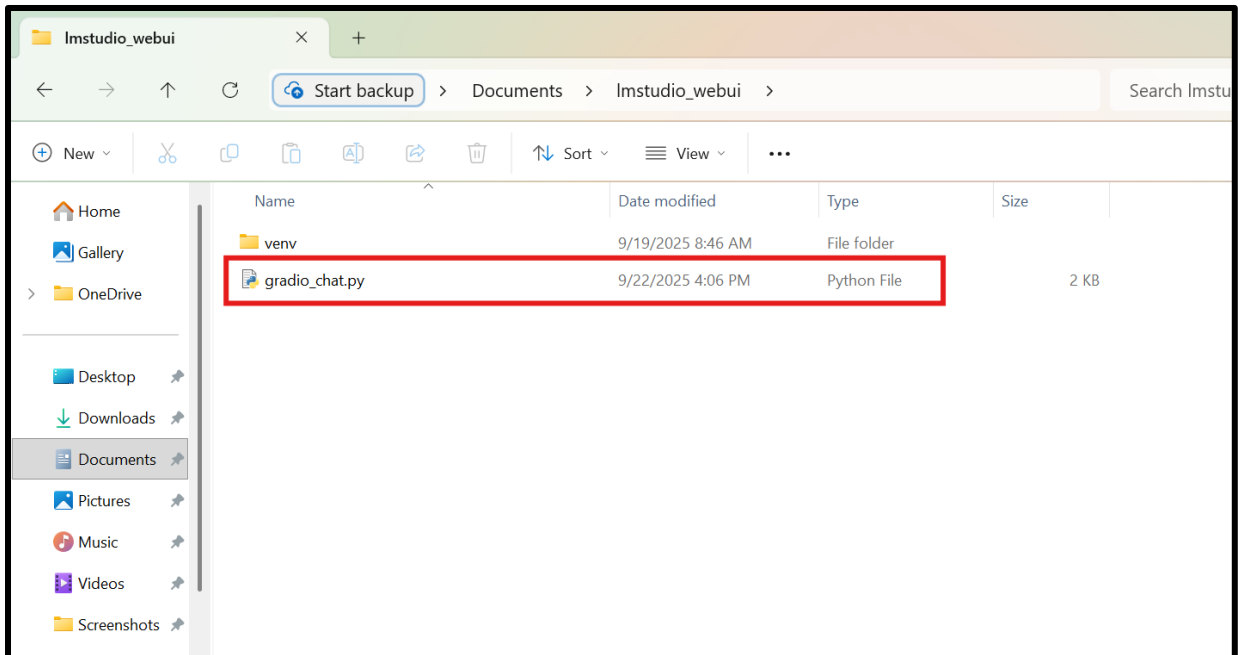
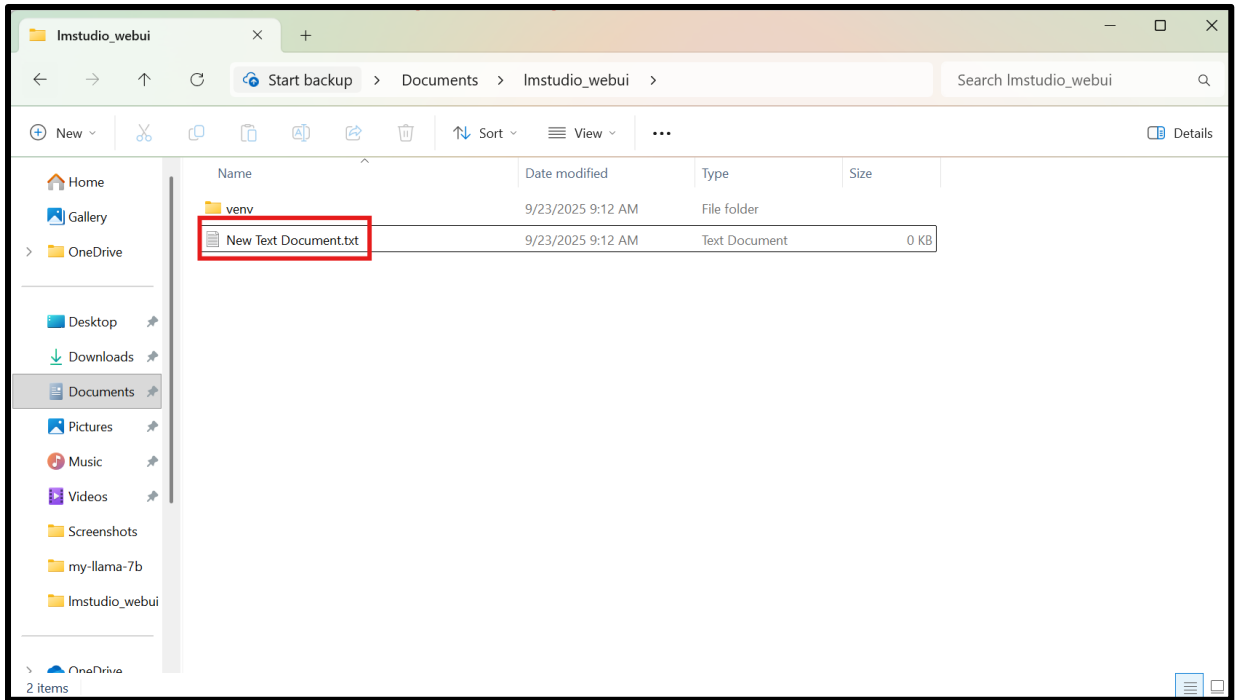
7. Verify LM Studio API & available models work

curl <http://insert ip address/v1/models>

Example: curl <http://192.168.40.58:8080/v1/models>

8. Create gradio_chat.py file inside lmstudio_webui & insert script.

Note: You can create a .py file by creating a text document and renaming it to gradio_chat.py. The icon for the document should change to the python logo afterwards.



gradio.py script (change “ip_address” to your ip address):

```
# -----  
# Import Gradio for building the web UI  
import gradio as gr  
  
# Import requests for sending HTTP requests to LM Studio  
import requests
```

```

# -----
# LM Studio API endpoint and model configuration

# URL for the LM Studio chat completions API
LMSTUDIO_URL = "http://ip_address:8080/v1/chat/completions"

# Name of the model to use
MODEL_NAME = "lmstudio-community/Meta-Llama-3-8B-Instruct"

# -----
# Function to send a message to LM Studio

def send_message(message, history):
    # HTTP headers specifying JSON format
    headers = {"Content-Type": "application/json"}

    # Build the payload containing model, chat history, and
    # settings
    payload = {
        "model": MODEL_NAME,
        "messages": [
            {"role": "system", "content": "You are a helpful
assistant."}
        ]
        + [
            {"role": "user", "content": m[0]} if i % 2 == 0
            else {"role": "assistant", "content": m[1]}
            for i, m in enumerate(history)
        ]
        + [{"role": "user", "content": message}],
        "temperature": 0.7 # randomness control
    }

    try:
        # Send a POST request to LM Studio with the payload
        response = requests.post(LMSTUDIO_URL, headers=headers,
json=payload, timeout=120)

        # Raise an error if the response status indicates a
        # failure
        response.raise_for_status()

        # Extract the assistant's reply from the JSON response
        reply =
response.json()["choices"][0]["message"]["content"]

```

```

except Exception as e:
    # Return an error message if something goes wrong
    reply = f"Error: {str(e)}"

    return reply

# -----
# Function to handle user input and chat history

def respond(message, chat_history):
    # Send message and history to the model
    reply = send_message(message, chat_history)

    # Append the new message and reply to chat history
    chat_history.append((message, reply))

    # Return empty textbox + updated chat history
    return "", chat_history

# -----
# Build the Gradio user interface

with gr.Blocks() as demo:
    # Title
    gr.Markdown("## Local LM Studio Chat")

    # Chat display component
    chatbot = gr.Chatbot(type="messages")

    # Input box for user messages
    msg = gr.Textbox(placeholder="Type your message...")

    # Button to clear chat history
    clear = gr.Button("Clear")

    # Connect input box submission to the respond function
    msg.submit(respond, [msg, chatbot], [msg, chatbot])

    # Connect clear button to reset chat
    clear.click(lambda: None, None, chatbot, queue=False)

# -----
# Launch the Gradio app

# server_name="0.0.0.0" allows access from other devices on your
network

```

```
# server_port=7860 is the port Gradio will use
demo.launch(server_name="0.0.0.0", server_port=7860)
```

9. Launch gradio webUI

```
python gradio_chat.py
```

10. Access the UI locally from PC & from phone. Change “ip_address” to the PC’s IP Address. Make sure your phone/other device is on the same network

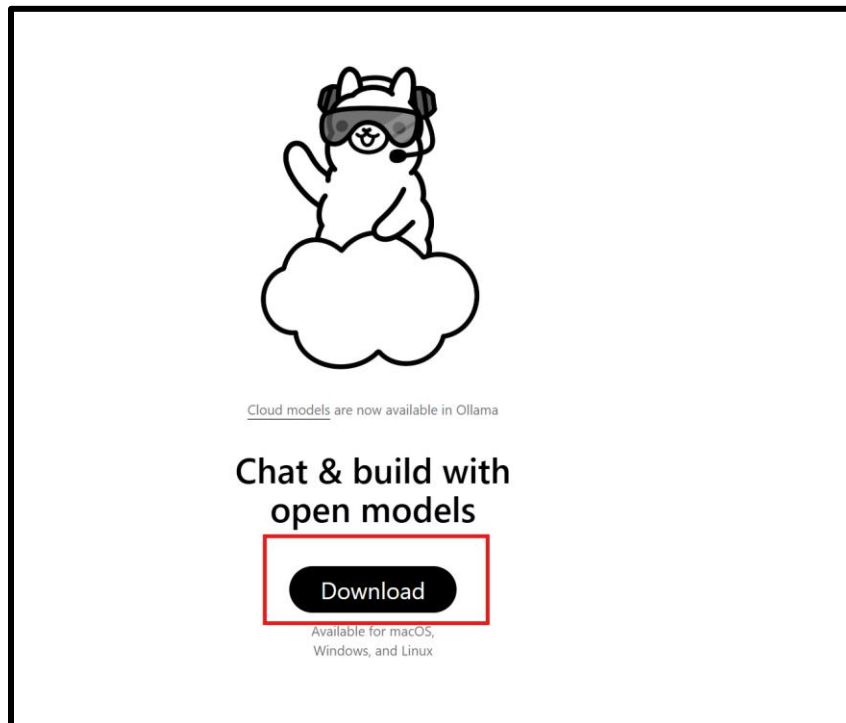
On your own PC, you can visit <http://127.0.0.1:7860>

On your phone/other device, you can visit http://ip_address:7860

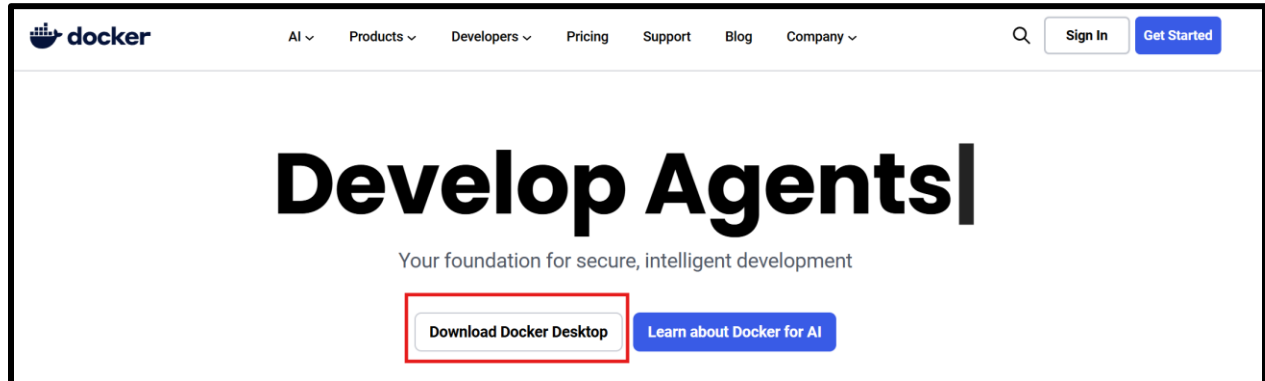
Lab Part 3: Create WebUI using Docker

1. Download Docker, Ollama, & Ubuntu

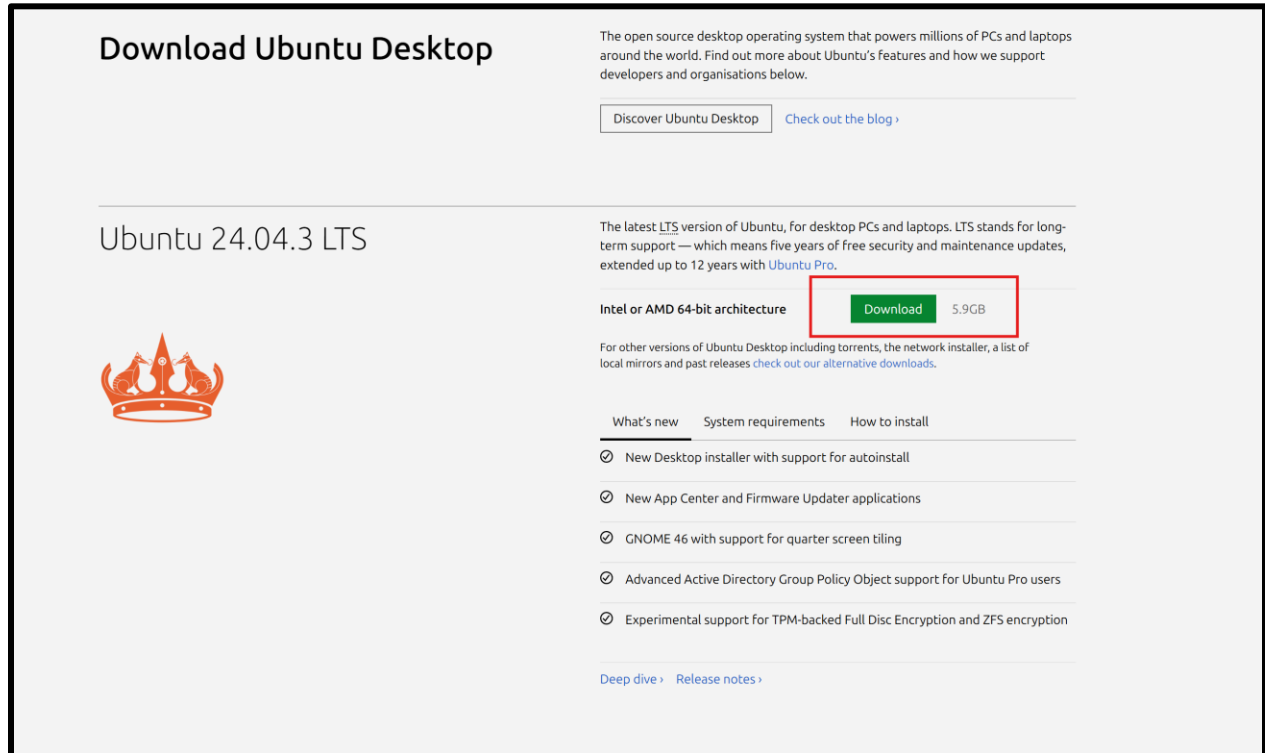
- <https://ollama.com/>



<https://www.docker.com/>



<https://ubuntu.com/download/desktop>



2. Run this script in Ubuntu/WSL Shell (single-line copy):

```
docker run -d \
  --name ollama \
  --restart unless-stopped \
  -v ollama:/root/.ollama \
  -p 11434:11434 \
```

```
-e OLLAMA_HOST=0.0.0.0:11434 \  
ollama/ollama:latest
```

Note: This command launches the Ollama Docker in a detached mode, then gives it a name, a restart policy, a persistent volume and port mapping. It also makes an environment variable so Ollama will listen to all network interfaces and runs the latest Ollama image.

3. Verify it's running

```
docker ps
```

4. Create a project folder for Open WebUI

```
cd ~ # Goes to home directory  
mkdir -p ollama-webui # Creates new directory "ollama-webui" in folder  
cd ollama-webui # Goes to said "ollama-webui"
```

5. Create docker-compose.yml & paste script

```
nano docker-compose.yml  
  
services:  
  openwebui:  
    image: ghcr.io/open-webui/open-webui:main  
    container_name: open-webui  
    volumes:  
      - open-webui:/app/backend/data  
    environment:  
      - OLLAMA_BASE_URL=http://host.docker.internal:11434  
      - WEBUI_AUTH=False  
    ports:  
      - "3000:8080"  
    restart: unless-stopped  
  
volumes:
```

```
open-webui:
```

Note: You can then save and exit nano by pressing CTRL+O, then Enter, then CTRL+X to exit

Note: These commands navigate to your home directory, create a new folder called ollama-webui, and then move into that folder. This sets up a dedicated workspace for the Ollama web UI project, so all related files are organized in one place.

6. Start Open WebUI

```
docker compose up -d
```

7. Open interfaces in a browser

You may visit the following links to open the API/Open WebUI. Replace ip_address with the machines ip address

- Ollama API: http://ip_address:11434

- Open WebUI: http://ip_address:3000

8. Pull a model into Ollama

Example: gemma3, 4b parameters

```
docker exec -it ollama ollama pull gemma3:4b
```

Problems:

1. Container name conflicts

Whenever I tried to start up the Open WebUI container I saw an error response that the container name ollama was already in use. I had to stop and remove an old ollama container before re-running by using the commands “docker stop ollama” and “docker rm ollama”

2. Open WebUI not reachable on PC or phone

When I first tried to reach the Open WebUI on my PC, I typed in http://localhost:3000, which didn't load the interface. I tried manually replacing local host with the PC's local ip address which fixed the issue. |

3. Difficulty editing docker-compose.yml in nano

Nano/Ubuntu has a very unintuitive editing interface with unordinary key binds. I also realized I couldn't click on a command but rather had to use my arrow keys to select one which was very tedious. I realized later that when I needed to remove a line in nano I just needed to repeatedly press CTRL+K to cut all lines then paste the new content carefully

4. File Extension Confusion

While doing my LM Studio AI Lab, I needed to create a file called `gradio_chat.py`, which is usually done by creating a text document and renaming it to `gradio_chat.py`. While I didn't realize it, I later found out that my LM Studio wouldn't work since it was still saved as `gradio_chat.py.txt` and was still a text document, not a python script.

5. HTTP 200 from port number 8080

I tried numerous times to load into LM Studio from my PC using port 8080 but then realized that port 8080 is the API server, not the Web UI, so I fixed this by accessing the gradio UI port which is 7860.

Conclusion:

Throughout this lab, I learned numerous skills. I had reviewed my python skills with gradio alongside software installation, but I also learned Linux, Ubuntu, and Docker skills. These skills are extremely valuable for Cybersecurity & IT which often use these skills for their work, but I also learned about LLM's, .gguf files, WebUI's, and parameters which are vital to any profession that uses AI for their work.